



Preface

This documentation is for version 1.4 of OpenX. This documentation is not final, and updates to both the documentation and program itself could change over time. The contents of this documentation follow the order in which they appear in the `openx.py` file. This documentation also assumes that you have imported OpenX as `openx` in Python. For reference, X goes left to right, Y goes up and down, Z goes front and back. “Length” corresponds to X, “width” corresponds with Z, and “height” corresponds to Y.

Non-Classified Functions

`openx.init()`

Set up OpenX for use by using this function to initialize everything, including the camera, mixer, lighting etc. Call this at the beginning of your main program before any other OpenX functions (except of course, `import openx`). Takes no parameters.

`openx.flip()`

Refreshes the screen, and renders whatever has been drawn to the screen. Call this function at the very end of your main loop. Takes no parameters.

`openx.load_texture(str filepath)`

Given the filepath for a texture, it will return the OpenGL `texture_id` for the image. This is only really used by OpenX to generate its own textures for Geometry, however you could accomplish more complicated tasks.

`openx.unload_texture(str filepath)`

Another function that unloads the texture from the texture cache generated from the above, to save up resources. Again, used mainly internally but could be used to achieve custom effects.

`openx.unload_texture(str filepath)`

Another function that unloads the texture from the texture cache generated from the above, to save up resources. Again, used mainly internally but could be used to achieve custom effects.

`openx.lights_on()`

Sets up the lighting effects for rendering geometry. Used internally, but could be used with the above texture functions to achieve custom effects. Takes no parameters.

`openx.lights_off()`

Unloads the lighting effects for rendering geometry. Used internally, but could be used with the above texture functions to achieve custom effects. Do this after rendering your custom geometry, and after using `openx.lights_on()`. Takes no parameters.

Controller Class

`openx.Controller.read()`

Takes no parameters. Returns a set of all the keys currently pressed on the keyboard as strings in all capitals. E.g. “LEFT SHIFT” would be included in a set. If no keys are being pressed, the method returns `set( )`. Also, when this method is called, and Mouse Look is enabled, read will capture the mouse if uncaptured.

`openx.Controller.set_sensitivity(float sensitivity)`

Takes in a float, which will set the sensitivity of the Mouse Look modes.

`openx.Controller.mode(int mode)`

Sets the mode of the controller. Any mode other than the ones listed will be interpreted as the default no-Mouse Look mode.

<i>int</i> mode	Description
1	Enables Mouse Look mode
2	Enables Mouse Look mode, but disables up and down movement

Camera Class

`openx.Camera.apply()`

Applies any current transforms in the camera caches. Any stored translations and rotations will be applied when this function is called. It is used internally, and not recommended to be used externally. Instead, use the `translate()` and `rotate()` functions. Takes no parameters.

`openx.Camera.translate(float x, float y, float z)`

Translates the camera in the direction that it's looking, given respective x, y, and z vectors (as floats). The bigger the value for x, y, and z, the farther the camera will move.

`openx.Camera.rotate(float yaw, float pitch, float roll=0)`

Rotates the camera given the yaw, pitch, and optionally roll in degrees (default is 0 degrees).

`openx.Camera.set_pos(float x, float y, float z)`

Sets the camera's current position to the given x, y, and z coordinates.

`openx.Camera.set_rotation(float yaw, float pitch, float roll=0)`

Sets the camera's rotation given the yaw, pitch, and optionally roll in degrees (default is 0 degrees).

`openx.Camera.get_rotation()`

Returns the camera's current rotation as an array containing the yaw, pitch and roll in that order. Takes no parameters.

`openx.Camera.get_pos()`

Returns the camera's current position as an array containing the x, y, and z coordinates in that order. Takes no parameters.

`openx.cull_rotation(array arr)`

Takes in an array and iterates through every value, and if it is above 360 it sets it to 0. Useful for if you are working with large sets of degrees.

Environment Class

`openx.Environment.set_fog_color(int r, int g, int b)`

Sets the scenes background and fog color to the specified RGB value given the ints r, g, and b.

Environment.Ceiling Class

`openx.Environment.Ceiling.draw()`

Draws the scene's ceiling. Useful for rendering skybox-like textures or level roofs. The Ceiling is located at $y = 6.5$.

`openx.Environment.Ceiling.apply_texture(string filepath)`

Applies a texture to the ceiling. Call this outside of your main loop to avoid reloading the texture over and over, which could tank performance.

Environment.Floor Class

`openx.Environment.Floor.draw()`

Draws the scene's ceiling. Useful for rendering skybox-like textures or level roofs. The Floor is located at $y = -6.5$.

`openx.Environment.Floor.apply_texture(string filepath)`

Applies a texture to the floor. Call this outside of your main loop to avoid reloading the texture over and over, which could tank performance.

Tex2D.Billboard Class

`openx.Tex2D.Billboard.__init__(self, float x, float y, float z, int width, int height, string filepath)`

Initializes a new billboard object, at the position (x, y, z) and with the width and height of *width* and *height*, with a texture *filepath*. Support PNG images and transparency (alpha channel).

`openx.Tex2D.Billboard.draw(self)`

Renders the billboard object in the scene. The size automatically updates based on the billboard's distance to the camera.

Geometry.RectangularPrism Class

`openx.Geometry.RectangularPrism.__init__(self, float l, float w, float h, float x, float y, float z)`

Creates a new Rectangular Prism geometry object in your scene, with length *l*, width *w*, height *h*, at *x*, *y*, and *z*.

`openx.Geometry.RectangularPrism.translate(self, float x, float y, float z)`

Performs a translation on the Rectangular Prism object, moving it *x* units in the *x* direction, and so forth for *y* and *z* as well.

`openx.Geometry.RectangularPrism.transform(self, float w, float h, float l)`

Transforms the scale/size of the Rectangular Prism object, by a new width, height and length respectively. **NOTE: In version 1.5, all methods that take l/w/h in that order will be reordered to be w/h/l instead.*

`openx.Geometry.RectangularPrism.rotate(self, float ax, float ay, float az)`

Rotates the Rectangular Prism objects by angle *ax* degrees on the *x* axis, *ay* on the *y* axis, and *az* degrees on the *z* axis.

`openx.Geometry.RectangularPrism.draw(self)`

Renders the geometry object to the screen to be drawn by `openx.flip()`.

`openx.Geometry.RectangularPrism.apply_texture(self, string filepath, int tiling=1)`

Applies a new texture, located at *filepath*, to the object. The optional *tiling* param specifies how many times the texture will tile on to the geometry object. It defaults to 1 (which is no tiling).

Geometry.TriangularPyramid Class

`openx.Geometry.TriangularPyramid.__init__(self, float l, float w, float h, float x, float y, float z)`

Creates a new Triangular Pyramid (a pyramid with a base that's a triangle) geometry object in your scene, with length *l*, width *w*, height *h*, at *x*, *y*, and *z*.

`openx.Geometry.TriangularPyramid.translate(self, float x, float y, float z)`

Performs a translation on the Triangular Pyramid object, moving it *x* units in the *x* direction, and so forth for *y* and *z* as well.

`openx.Geometry.TriangularPyramid.transform(self, float w, float h, float l)`

Transforms the scale/size of the Triangular Pyramid object, by a new width, height and length respectively.

**NOTE: In version 1.5, all methods that take l/w/h in that order will be reordered to be w/h/l instead.*

`openx.Geometry.TriangularPyramid.rotate(self, float ax, float ay, float az)`

Rotates the Triangular Pyramid objects by angle *ax* degrees on the *x* axis, *ay* on the *y* axis, and *az* degrees on the *z* axis.

`openx.Geometry.TriangularPyramid.draw(self)`

Renders the geometry object to the screen to be drawn by `openx.flip()`.

`openx.Geometry.TriangularPyramid.apply_texture(self, string filepath, int tiling=1)`

Applies a new texture, located at *filepath*, to the object. The optional *tiling* param specifies how many times the texture will tile on to the geometry object. It defaults to 1 (which is no tiling).

Geometry.Pyramid Class

`openx.Geometry.Pyramid.__init__(self, float l, float w, float h, float x, float y, float z)`

Creates a new Pyramid (a pyramid with a base that's a square) geometry object in your scene, with length *l*, width *w*, height *h*, at *x*, *y*, and *z*.

`openx.Geometry.Pyramid.translate(self, float x, float y, float z)`

Performs a translation on the Pyramid object, moving it *x* units in the *x* direction, and so forth for *y* and *z* as well.

`openx.Geometry.Pyramid.transform(self, float w, float h, float l)`

Transforms the scale/size of the Pyramid object, by a new width, height and length respectively.

**NOTE: In version 1.5, all methods that take l/w/h in that order will be reordered to be w/h/l instead.*

`openx.Geometry.Pyramid.rotate(self, float ax, float ay, float az)`

Rotates the Pyramid objects by angle *ax* degrees on the *x* axis, *ay* on the *y* axis, and *az* degrees on the *z* axis.

`openx.Geometry.Pyramid.draw(self)`

Renders the geometry object to the screen to be drawn by `openx.flip()`.

`openx.Geometry.Pyramid.apply_texture(self, string filepath, int tiling=1)`

Applies a new texture, located at *filepath*, to the object. The optional *tiling* param specifies how many times the texture will tile on to the geometry object. It defaults to 1 (which is no tiling).

Geometry.Sphere Class

`openx.Geometry.Sphere.__init__(self, float r, float x, float y, float z)`

Creates a new sphere geometry object in your scene, with a radius of *r* units, at *x*, *y*, and *z*.

`openx.Geometry.Sphere.translate(self, float x, float y, float z)`

Performs a translation on the Sphere object, moving it *x* units in the *x* direction, and so forth for *y* and *z* as well.

`openx.Geometry.Sphere.transform(self, float r)`

Transforms the length of the radius of the sphere.

`openx.Geometry.Sphere.rotate(self, float ax, float ay, float az)`

Rotates the sphere object's by angle *ax* degrees on the *x* axis, *ay* on the *y* axis, and *az* degrees on the *z* axis.

`openx.Geometry.Sphere.draw(self)`

Renders the geometry object to the screen to be drawn by `openx.flip()`.

`openx.Geometry.Sphere.apply_texture(self, string filepath)`

Applies a new texture, located at *filepath*, to the object. Tiling is **not** available on quadric shapes (spheres, cylinders, and disks) in the current version of OpenX.

Geometry.Cylinder Class

`openx.Geometry.Cylinder.__init__(self, float b, float t, float h, float x, float y, float z)`

Creates a new cylinder geometry object in your scene, with a bottom radius of *b* units, a top radius of *t* units, at *x*, *y*, and *z*. Please note that the top and bottom disks of the cylinder are not rendered, just the sides. By specifying a top and bottom radius, it is possible to create cones using this class.

`openx.Geometry.Cylinder.translate(self, float x, float y, float z)`

Performs a translation on the Cylinder object, moving it *x* units in the *x* direction, and so forth for *y* and *z* as well.

`openx.Geometry.Cylinder.transform(self, float b, float t, float h)`

Transforms the bottom radius, top radius, and height of the cylinder.

`openx.Geometry.Cylinder.rotate(self, float ax, float ay, float az)`

Rotates the cylinder object's by angle *ax* degrees on the x axis, *ay* on the y axis, and *az* degrees on the z axis.

`openx.Geometry.Cylinder.draw(self)`

Renders the geometry object to the screen to be drawn by `openx.flip()`.

`openx.Geometry.Cylinder.apply_texture(self, string filepath)`

Applies a new texture, located at *filepath*, to the object. Tiling is **not** available on quadric shapes (spheres, cylinders, and disks) in the current version of OpenX.

Geometry.Disk Class

`openx.Geometry.Disk.__init__(self, float i, float o, float h, float x, float y, float z)`

Creates a new disk geometry object in your scene, with an inner radius of *i* units, an outer radius of *o* units, at *x*, *y*, and *z*. The inner radius specifies the radius of a hole to be made in the overall disk. If *i* is 0.0, then it will look like a regular disk. If $0.0 < i < o$, then it will look like a donut-esque shape.

`openx.Geometry.Disk.translate(self, float x, float y, float z)`

Performs a translation on the Disk object, moving it *x* units in the *x* direction, and so forth for *y* and *z* as well.

`openx.Geometry.Disk.transform(self, float i, float o)`

Transforms the inner and outer radii of the disk.

`openx.Geometry.Disk.rotate(self, float ax, float ay, float az)`

Rotates the disk object's by angle *ax* degrees on the x axis, *ay* on the y axis, and *az* degrees on the z axis.

`openx.Geometry.Disk.draw(self)`

Renders the geometry object to the screen to be drawn by `openx.flip()`.

`openx.Geometry.Disk.apply_texture(self, string filepath)`

Applies a new texture, located at *filepath*, to the object. Tiling is **not** available on quadric shapes (spheres, cylinders, and disks) in the current version of OpenX.

Font Class

`openx.Font.draw_rgba(int x, int y, string text, quadruple color, quadruple bg_color)`

Draws text with a solid background color, at *x* and *y* on the screen. Please note that this refers to pygame screen coordinates: not 3D coordinates. The text foreground color is determined by the quadruple *color*, as an RGBA value expressed as (R, G, B, A). The same goes with *bg_color*, which determines the background color. Every time this method is called, it stores the text in a font cache, so please use `draw_dynamic()` for text that changes very often, to avoid enlarging the cache excessively, which can hurt performance.

`openx.Font.draw(int x, int y, string text, quadruple color,)`

Same as `openx.Font.draw_rgba()` but omits the background color and renders the text with no background (transparent).

`openx.Font.draw_dynamic(int x, int y, string text, quadruple color,)`

Same as `openx.Font.draw()`, but does not add the text into a cache. Useful for text that updates constantly or very often, such as FPS counters or scores. For one-time text renders, it is more resource efficient to use `draw()` or `draw_rgba()`.

XObject Class

How do I make XObject (.xobj) files?

XObject files, known with the extension of *.xobj*, allows users to create custom level geometry, with the same capabilities as regular `openx.Geometry` objects. There is no editor to create them currently, so you'll have to draw and plan them by hand. Here is the internal structure of an *.xobj* file. You can simply write these in a .txt file in an IDE or Notepad and rename the extension to *.xobj* to make it work.

Vertices are defined with the prefix *v*. They take three inputs: the relative *x*, *y*, and *z* position of the vertices as floats. Here is an example of a vertex defined at the center of the XObject:

```
v 0 0 0
```

Vertices, when created, are assigned an internal id based on the order you wrote them in, starting at **1**, not 0. Thus, faces can be defined as a set of four vertices (triangular faces are not yet supported) using the **f** prefix. Given four vertex IDs, it can define a new face.

```
v 0 0 0 //ID = 1
v 0 1 0 //ID = 2
v 1 0 1 //ID = 3
v 1 1 1 //ID = 4
f 1 2 3 4
```

openx.XObject.__init__(*self*, string filepath, float x, float y, float z, float l, float w, float h)

Creates a new XObject object. Given the *filepath* of the .xobj file, and its x, y, z coordinates and its length, width, and height.

openx.XObject.translate(*self*, float x, float y, float z)

Performs a translation on the XObject object, moving it *x* units in the *x* direction, and so forth for *y* and *z* as well. Rotating and scaling are not yet supported in XObjects.

openx.XObject.draw(*self*)

Renders the geometry object to the screen to be drawn by openx.flip().

openx.XObject.draw(*self*, string filepath, int tiling=1)

Applies a new texture, located at *filepath*, to the object. The optional *tiling* param specifies how many times the texture will tile on to the geometry object. It defaults to 1 (which is no tiling).

Audio.Mixer Class

Note About CodeHS and Audio

The Audio class, which includes Audio.Mixer and Audio.SFX do not work on CodeHS due to their servers / IDE not supporting audio implementation. If you do not load the `DEBUG_CODEHS_MODE()` function after `init()`, then this will cause OpenX to crash every boot.

`openx.Audio.Mixer.load(string filepath)`

Loads an .ogg (required) file for playback in your scene. Note: this function will not play your file, it just loads it. Use `openx.Audio.Mixer.play()` to play your songs.

`openx.Audio.Mixer.unload()`

Unloads all songs in the Mixer to relieve resources.

`openx.Audio.Mixer.play(int loops=1, float start=0.0, int fade=0)`

Plays the loaded audio files in the mixer. Optional param *loops* determine how many times to play the song. Loading *loops* as 0 will make the song play endlessly. Optional param *start* defines how late in seconds to start the song from. Optional param *fade* determines how long a fade in should be for the song in milliseconds, if there should be more than 0.

`openx.Audio.Mixer.rewind()`

Rewinds the loaded song(s) in the mixer back to the start.

`openx.Audio.Mixer.stop()`

Stops all playback, meaning it rewinds the song as well. This does not pause playback: see `openx.Audio.Mixer.pause()` and `openx.Audio.Mixer.unpause()`.

`openx.Audio.Mixer.pause()`

Pauses current mixer song.

`openx.Audio.Mixer.unpasue()`

Unpauses current mixer song, if paused.

`openx.Audio.Mixer.fadeout(float time)`

Fades the current song out in *time* seconds.

`openx.Audio.Mixer.set_volume(float vol)`

Sets the mixer's volume. *vol* is in a range from 0.0 to 1.0.

`openx.Audio.Mixer.get_volume()`

Returns the mixer's current volume.

`openx.Audio.Mixer.busy()`

Returns *True* if the Mixer is busy, *False* otherwise.

`openx.Audio.Mixer.seek(float timestamp)`

Sets the current mixer songs to seek to the specific point at *timestamp*, in seconds.

`openx.Audio.Mixer.queue(string filepath, int loops=1)`

Adds a new .ogg file to the queue for the mixer. Optional param *loops* determines how many times to play the song. A value of 0 makes it play endlessly.

Audio.SFX Class

`openx.Audio.SFX.__init__(self, string filepath, float x=0, float y=0, float z=0, float d=0)`

Creates a new sound effect object with the audio file path of *filepath*. Optionally you may set the position of the sound to play at using *x*, *y* and *z*. The param *d* is the distance at which the sound from its origin will be silent. This is not needed if your sound effect is not meant to deteriorate with distance.

`openx.Audio.SFX.play(self, int loops=1, int fade=0)`

Plays a sound effect object. Optional param *loops* determines how many times to play the sound effect. Load 0 to make it loop endlessly. Optional param *fade* determines how long to fade in the sound effect in milliseconds.

`openx.Audio.SFX.stop(self, int fade=0)`

Stops a sound effect from playing. Optional param *fade* determines how long to fade out the sound effect in milliseconds.

`openx.Audio.SFX.set_volume(self, float vol)`

Sets the volume of the sound effect in a range of 0.0 to 1.0.

openx.Audio.SFX.get_volume(*self*)

Returns the current volume level of a sound effect object.

openx.Audio.SFX.length(*self*)

Returns the length of the audio sound effect.

openx.Audio.SFX.calculate_att(*self*)

Calculates the attenuation for a sound effect (determine the volume based on how far away the sound is). When called, this method automatically changes the volume of the sound effect based on the initialized *x*, *y*, *z*, and *d* values.

Collision.Hitbox Class

openx.Collision.Hitbox.__init__(*self*, float *l*, float, *w*, float *h*, float *x*, float *y*, float *z*)

Creates a new hitbox with length, width and height of *l*, *w*, and *h*, at *x*, *y*, and *z*. Once a hitbox is created, it is added to the hitbox cache.

openx.Collision.Hitbox.translate(*self*, float *x*, float *y*, float *z*)

Moves the hitbox in world space by *x* units in the *x* axis, and so forth for *y* and *z*.

openx.Collision.Hitbox.transform(*self*, float *w*, float, *h*, float *l*)

Transforms a hitbox's width (*w*), height (*h*), and length (*l*).

openx.Collision.Hitbox.rotate(*self*, float *ax*, float, *ay*, float *az*)

Rotates a hitbox object's boundaries, by *ax* degrees around the *x*-axis, and so forth for *ay* and *az*.

openx.Collision.Hitbox.collides_with(*self*, Collision.Hitbox *other*)

Returns *True* if the hitbox object is colliding with another hitbox object, *other*.

openx.Collision.Hitbox.hits_barrier(*self*)

Returns *True* if this hitbox object is colliding with any other hitbox object, regardless of its ID in the hitbox cache.

`openx.Collision.Hitbox.draw(self, tuple color=(0,1,0) )`

Draws a hitbox object's boundary, useful for troubleshooting/debugging. Optional param *color*, which is a tuple, to determine the RGB value of the color of the bounding box. Defaults to green. Note: the color value is from 0.0 to 1.0, not 0 to 255.

Collision.Hitbox Class

`openx.Debug.get_fps()`

Returns the current FPS as an int.

`openx.Debug.get_mem()`

Returns the current amount of free memory in GB. Always rounds to two decimal places.

Getting OpenX to work on CodeHS

`openx.DEBUG_CODEHS_MODE()`

In order to get OpenX to run efficiently with CodeHS, you need to call this function after `openx.init()`, but before anything else. This function sets the resolution to be 240p (still in 4:3) scaled up two times, to render at 480p.

If you want your application to run smoothly in the CodeHS runtime environment you must also use as many optimization techniques as possible. For example, by default on CodeHS, all textures preloaded by OpenX (like the missing and error textures) are capped at 128*128px, while on PC they are 512*512px. Running OpenX natively in Python on a Windows machine allows for peak performance and textures well above the 512*512px limit.

If you wish to download OpenX for use on PC, you can download it at <https://github.com/AMWOnline/OpenX>.

Setting up your first file

```
import openx

openx.init()

def main():
    while True:

        keys = openx.Controller.read()

        openx.flip()

main()
```

To set up a new OpenX file, create a new .py file (name it whatever you want). Import OpenX. Every release of OpenX from GitHub will always use the name `openx.py`. Then, make sure you initialize OpenX using the `init()` function. You don't technically need a main function, but it does provide some specific use cases when your main loop is in a function you can call later.

In your main function, in order to get the keys that are currently pressed down (to of course perform functions in your scene).

At the end of your main loop, make sure you use `flip()` so that way the screen refreshes every frame.